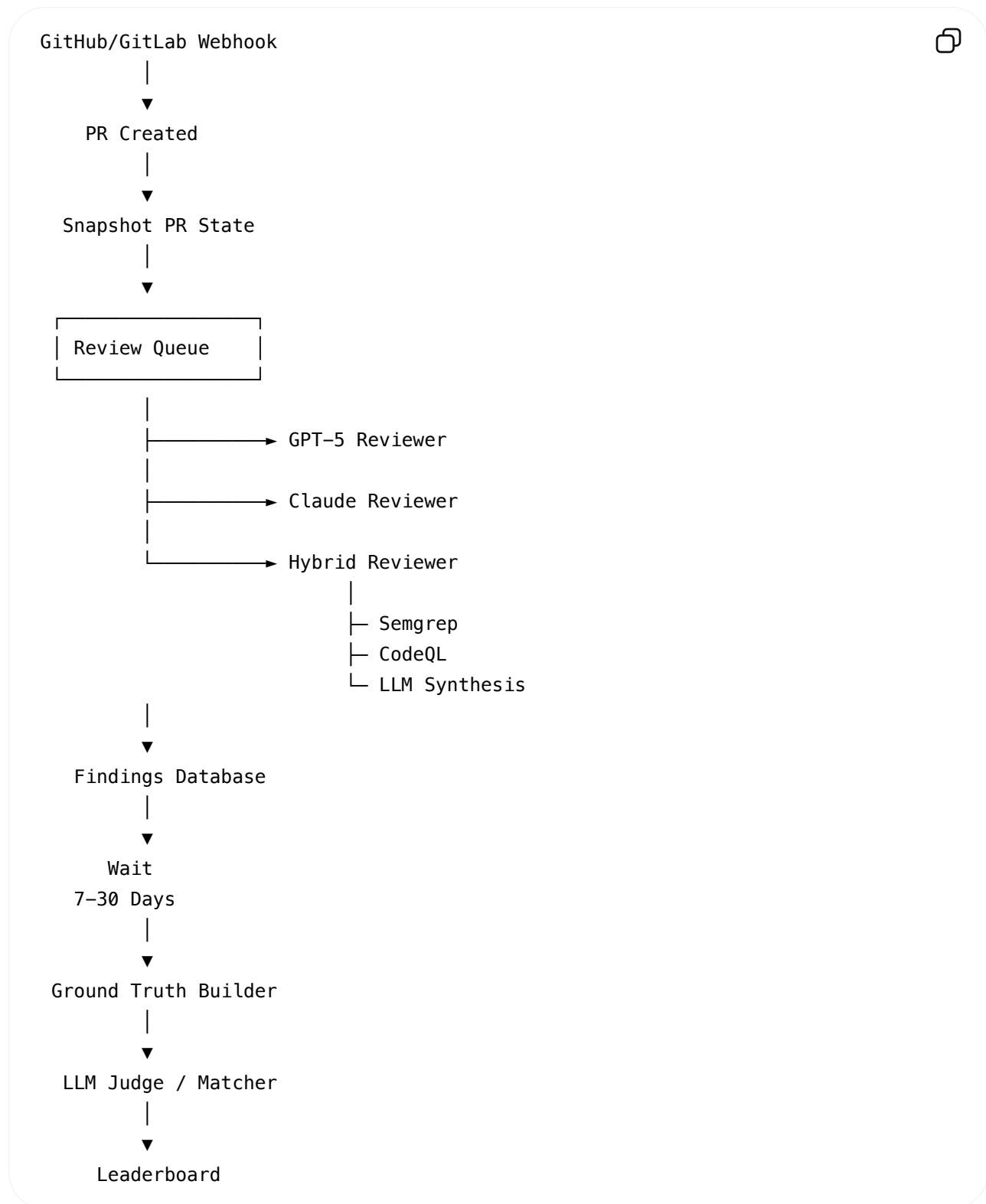


If you're building an **online benchmark** for private GitHub/GitLab repositories and your reviewers are limited to:

1. GPT-5
2. Claude
3. Hybrid (Static Analysis + LLM)

then I would design it as a continuous evaluation platform rather than a one-time benchmark.

High-Level Architecture



Phase 1: Capture the PR

When a PR opens:

Store:

 JSON 

```
{
  "pr_number": 1234,
  "repo": "backend-service",
  "base_commit": "...",
  "head_commit": "...",
  "diff": "...",
  "files_changed": [...]
}
```

For GitHub:

- Pull Request webhook
- Pull Request Review webhook
- Push webhook

For GitLab:

- Merge Request webhook
- Note webhook
- Push webhook

The critical piece is preserving the exact PR state before human review begins.

Phase 2: Run Review Agents

GPT-5 Agent

Input:

System:

You are a senior software engineer.



Review this PR for:

- correctness
- security
- maintainability
- performance

Return findings as JSON.

Output:

</> JSON



```
[
  {
    "file": "auth.py",
    "line": 45,
    "severity": "high",
    "title": "Potential authorization bypass"
  }
]
```

Store everything.

Claude Agent

Use the identical format.

The benchmark should normalize outputs so you're comparing findings, not formatting.

Phase 3: Hybrid Agent

This is the configuration I'd expect to win.

Step 1

Run Semgrep.

Example:

</> Bash



```
semgrep scan .
```

Collect:

</> JSON



```
[
  {
    "rule":"python.lang.security",
    "file":"auth.py",
    "line":45
  }
]
```

Step 2

Run CodeQL.

</> Bash



```
codeql database analyze
```

Collect results.

Step 3

Give static analysis results to GPT-5.

Prompt:

PR Diff:



...

Semgrep Findings:

...

CodeQL Findings:

...

Review the PR.

Deduplicate findings.

Find additional issues not detected
by static analysis.

The LLM becomes a synthesizer plus a reasoning engine.

This almost always performs better than LLM-only review.

Phase 4: Build Ground Truth

This is the hardest part.

The Martian benchmark uses accepted comments.

For internal repos, I'd use multiple signals.

Signal 1

Human review comments

Reviewer:
Null pointer possible here.



Ground truth candidate.

Signal 2

Follow-up commits

Fix race condition in cache layer



Ground truth candidate.

Signal 3

Reverted commits

Revert PR #1234



Strong signal.

Signal 4

Hotfixes

Production fix for PR #1234



Very strong signal.

Signal 5

Incidents

Link:

Incident-2026-115



back to:

PR #1234



Highest quality signal.

Phase 5: Automated Matching

Now compare:

Agent finding:

Missing null check on user.email



versus:

Developer fix:

Added validation for user.email



Use an LLM judge.

Prompt:

Finding:

...

Developer Fix:

...

Did the finding predict
the issue fixed by the developer?

Answer:

YES / NO

Confidence:

0-100



Store results.

Phase 6: Score Everything

For every agent:

Precision

Valid Findings

All Findings



Recall

Matched Ground Truth Issues

All Ground Truth Issues



Severity Recall

Weight:

Critical = 10

High = 5

Medium = 2

Low = 1



Then score accordingly.

The Most Practical MVP

If I were implementing this today, I'd start with:

Phase 1

- ✓ GitHub/GitLab webhook ingestion

Phase 2

- ✓ GPT-5 reviewer
- ✓ Claude reviewer
- ✓ Semgrep

Phase 3

- ✓ Hybrid reviewer

Phase 4

- ✓ Human review comments
- ✓ Follow-up commits

Phase 5

- ✓ GPT-5 judge

Phase 6

- ✓ Leaderboard



and defer:

- CodeQL (heavier operationally)
- Incident tracking
- Production defect linkage
- Long-term outcome scoring

until later.

A Better Enterprise Variant

One improvement I'd make over the original Martian approach is to evaluate **before posting comments to developers**.

Run the agents silently:

PR Created



GPT-5 Review

Claude Review

Hybrid Review



Store Results



Humans Review Normally



Benchmark Scoring



This avoids benchmark contamination, where developers start fixing issues because an AI suggested them. It lets you measure how well each agent predicts what experienced reviewers eventually identify and fix in the normal review process.

For a private GitHub Enterprise or GitLab environment, that silent-review architecture is usually the cleanest path to obtaining statistically meaningful precision and recall metrics over hundreds or thousands of real PRs.



Darktrace · Sponsored



AI is outpacing governance. Start securing AI.

Gain visibility into AI systems and close security gaps as new risks emerge. Get the latest guide



Before writing code, there is an important design decision:

What exactly should the GitLab collector collect?

For the offline benchmark architecture we discussed, a collector typically needs to gather:

1. Merge Request metadata
2. MR diffs
3. Changed files
4. Discussion threads
5. Review comments
6. Approvals
7. Commits
8. Post-review commits
9. Merge information
10. Author/reviewer information
11. Optional labels and milestones

A production collector is usually a few hundred lines of code because GitLab spreads this information across multiple APIs.

A good approach is to build it in stages:

Stage 1 (MVP)

Collect:

- Merge Request metadata
- Diffs
- Discussions
- Commits

Output:

JSON



```
{
  "project_id": 123,
  "mr_id": 456,
  "title": "...",
  "description": "...",
  "created_at": "...",
  "merged_at": "...",
  "diff": "...",
  "commits": [],
  "discussions": []
}
```

Stage 2

Add:

- Approvals
- Reviewers
- Labels
- Pipelines

Stage 3

Add:

- Ground-truth extraction
- Snapshot generation
- Incremental sync

A solid MVP collector might look like this:

Python



Run

```

from __future__ import annotations

import json
import os
from pathlib import Path
from typing import Any

import requests

class GitLabCollector:
    """
    Offline benchmark collector for GitLab Merge Requests.

    Collects:
    - MR metadata
    - Changes / diff
    - Discussions
    - Commits
    """

    def __init__(
        self,
        base_url: str,
        private_token: str,
        output_dir: str = "dataset",
    ):
        self.base_url = base_url.rstrip("/")
        self.output_dir = Path(output_dir)

        self.session = requests.Session()
        self.session.headers.update(
            {"PRIVATE-TOKEN": private_token}
        )

    def _get(self, endpoint: str, params: dict | None = None) -> Any:
        url = f"{self.base_url}/api/v4{endpoint}"

        response = self.session.get(
            url,
            params=params,
            timeout=60,
        )

        response.raise_for_status()
        return response.json()

    def get_merge_request(
        self,
        project_id: int,
        mr_iid: int,
    ) -> dict:
        return self._get(

```

```

        f"/projects/{project_id}/merge_requests/{mr_iid}"
    )

    def get_changes(
        self,
        project_id: int,
        mr_iid: int,
    ) -> dict:
        return self._get(
            f"/projects/{project_id}/merge_requests/{mr_iid}/changes"
        )

    def get_discussions(
        self,
        project_id: int,
        mr_iid: int,
    ) -> list:
        return self._get(
            f"/projects/{project_id}/merge_requests/{mr_iid}/discussions"
        )

    def get_commits(
        self,
        project_id: int,
        mr_iid: int,
    ) -> list:
        return self._get(
            f"/projects/{project_id}/merge_requests/{mr_iid}/commits"
        )

    def collect_merge_request(
        self,
        project_id: int,
        mr_iid: int,
    ) -> dict:

        mr = self.get_merge_request(project_id, mr_iid)

        changes = self.get_changes(project_id, mr_iid)

        discussions = self.get_discussions(
            project_id,
            mr_iid,
        )

        commits = self.get_commits(
            project_id,
            mr_iid,
        )

        snapshot = {
            "project_id": project_id,
            "mr_iid": mr_iid,

```

```

        "mr_iid": mr_iid,
        "metadata": mr,
        "changes": changes,
        "discussions": discussions,
        "commits": commits,
    }

    return snapshot

def save_snapshot(
    self,
    snapshot: dict,
) -> Path:

    project_id = snapshot["project_id"]
    mr_iid = snapshot["mr_iid"]

    target_dir = (
        self.output_dir
        / str(project_id)
        / str(mr_iid)
    )

    target_dir.mkdir(
        parents=True,
        exist_ok=True,
    )

    file_path = target_dir / "snapshot.json"

    with open(
        file_path,
        "w",
        encoding="utf-8",
    ) as f:
        json.dump(
            snapshot,
            f,
            indent=2,
            ensure_ascii=False,
        )

    return file_path

```

Example usage:

Python



Run

```

collector = GitLabCollector(
    base_url="https://gitlab.company.com",
    private_token=os.environ["GITLAB_TOKEN"],
)

snapshot = collector.collect_merge_request(
    project_id=123,
    mr_iid=456,
)

collector.save_snapshot(snapshot)

```

For a serious offline benchmark, I would actually recommend going one step further and storing each MR as a **normalized benchmark record** rather than raw GitLab JSON. Something like:

</> JSON



```

{
  "benchmark_id": "project123_mr456",
  "repository": "payments-service",
  "merge_request": {
    "iid": 456,
    "title": "...",
    "description": "..."
  },
  "files": [...],
  "diff": "...",
  "review_comments": [...],
  "commits": [...],
  "ground_truth": []
}

```

That normalized format makes it much easier later to support both GitLab and GitHub with the same reviewer, judge, and scoring pipelines.

